

COMPUTER SCIENCE PROJECT SYNOPSIS

- S.RAGHUVVEER 11B

Title of the Project:

CycloRoute – Intelligent Cycling Route Analysis and Recommendation System

Objective:

- To develop an intelligent and user-friendly desktop application for cycling-route analysis.
- To automatically collect route, elevation, weather, wind and rainfall information from suitable open/free data sources.
- To analyze the physical effects of road gradient, elevation, rolling resistance and wind on cycling.
- To calculate relative wind velocity and direction with respect to the cyclist.
- To analyze routes segment by segment.
- To incorporate optional user-provided local knowledge for conditions unavailable through external data sources.
- To compare alternative routes using physical and environmental factors.
- To provide an automated route recommendation.
- To investigate machine learning techniques for improving route predictions when sufficient data is available.
- To develop a professional graphical user interface for displaying maps, route information, graphs and recommendations.

To implement the application using modular Python programming and automated testing.

Introduction:

Cycling is influenced by considerably more than the distance between two locations. Factors such as road gradient, elevation, wind, rainfall, weather and road conditions can affect both the difficulty and suitability of a cycling route. A route that is shorter according to conventional navigation may not necessarily be the most suitable route for a cyclist under current environmental conditions.

CycloRoute is proposed as an intelligent desktop application that combines route information, live environmental data and physics-based analysis to evaluate cycling routes. The application

will obtain available information automatically and divide a journey into individual route segments so that changing conditions can be considered throughout the journey.

The physics component will calculate factors such as gravitational resistance, rolling resistance and relative wind velocity. Wind speed and direction will be converted into velocity vectors and compared with the cyclist's velocity to determine the relative wind experienced by the cyclist.

Environmental information such as rainfall, temperature and weather will be incorporated into the route analysis. Since online databases may not contain every local condition, particularly in rural areas, CycloRoute will also allow users to contribute local knowledge such as flood-prone, waterlogged or rain-sensitive road sections.

The project will also investigate machine-learning techniques where sufficient historical data is available. The aim will be to determine whether machine learning can improve predictions such as route difficulty or travel conditions.

A professional graphical user interface will present maps, route comparisons, environmental information, graphs, calculated physical factors and route recommendations.

CycloRoute therefore combines Physics, Mathematics, Python Programming, API Integration, Data Analysis, Database Management, GUI Development and Machine Learning to create a practical real-world cycling analysis system.

Problem Definition:

Cyclists are affected by several factors while traveling, including road gradient, elevation, wind, rainfall, weather and road surface conditions. Conventional navigation systems mainly focus on finding a route between two locations and may not analyze these factors specifically from a cyclist's perspective.

This problem is more significant in rural areas, where important local information such as flood-prone roads, waterlogged sections, poor road surfaces and other weather-sensitive conditions may not be available through online mapping services.

CycloRoute aims to address this problem by combining route information, live environmental data, physics-based calculations and user-provided local knowledge to analyze cycling routes and recommend a suitable route.

Project Details:

This project proposes the development of CycloRoute, a Python-based desktop application that analyses and compares cycling routes using freely available and open data sources.

The user will primarily enter a starting point and destination. The application will automatically obtain available route, elevation, weather, wind and rainfall information through suitable APIs and process the journey as individual route segments.

- Obtain route and geographic information using OpenStreetMap-based open data and services.
- Analyze road gradient and elevation along the route.
- Obtain live weather information including temperature, rainfall/precipitation, humidity and other available environmental parameters.
- Obtain live wind speed and direction.
- Calculate the effect of wind relative to the cyclist's direction using vector mathematics.
- Calculate gravitational resistance and rolling resistance.
- Analyses the effects of headwind, tailwind and crosswind.
- Analyses changing conditions across different route segments.
- Allow users to provide optional local knowledge that may not be available through online services.
- Allow local conditions such as flood-prone roads, water-logging, rain-sensitive sections and poor road conditions to be recorded.
- Compare alternative cycling routes using physical and environmental factors.
- Recommend a suitable route based on the available information.
- Store relevant local and historical information for future analysis.
- Investigate machine-learning techniques for improving predictions when sufficient data is available.
- Display route information, environmental conditions, calculations, graphs and recommendations through a professional graphical user interface.

The application will be designed so that information normally available from external sources is automatically collected rather than manually entered by the user.

Data sources for machine learning:

CycloRoute will generate its own dataset over time. It avoids relying on existing external datasets.

- Every time a user analyzes a route, the app logs segment-level data. This includes gradient, wind speed, direction, temperature, rainfall, road-surface info, and calculated physical factors.

- The app stores this data in a local SQLite database.
- As users continue to use the app, this history forms the dataset for training and evaluating machine-learning models.
- In early stages, limited historical data may restrict prediction effectiveness.
- Therefore, core route analysis and recommendations rely primarily on physics-based calculations. These do not depend on historical data.
- Machine learning will serve as an additional enhancement once sufficient data is collected.

Reason for Choosing the Topic:

- Cycling is significantly affected by physical and environmental conditions.
- Combines Physics, Mathematics, Python Programming, APIs, Data Analysis, Databases and Machine Learning.
- Demonstrates the real-world application of vector mathematics and mechanics.
- Addresses limitations of conventional route-planning systems from a cyclist's perspective.
- Provides practical experience in working with live environmental data.
- Considers local information that may not be available in digital mapping databases.
- Demonstrates the development of a complete desktop application with a professional GUI.
- Provides an opportunity to explore machine learning for predictive route analysis.
- Uses open and freely available data sources wherever practical.

Important Formula used:

Aerodynamic Drag formula:

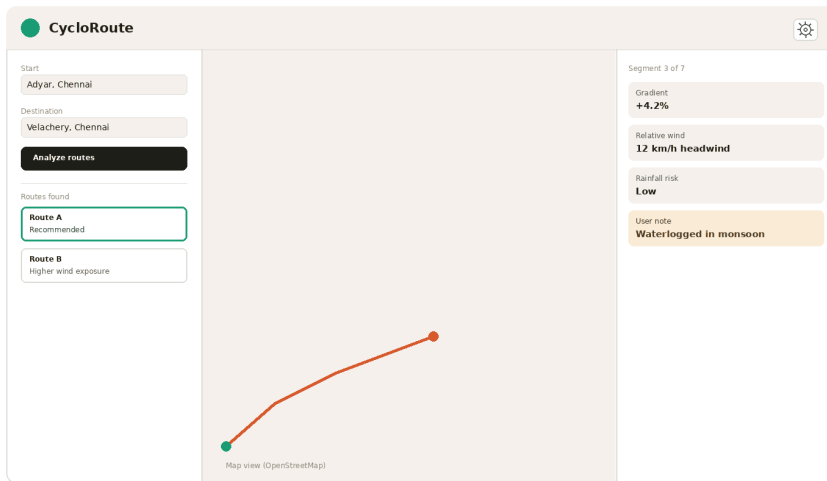
$$F_{aero} = \frac{1}{2} \rho C_d A v_{rel}^2$$

Total resistive power required to maintain speed:

$$P = v (F_{gravity} + F_{rolling} + F_{aero})$$

Proposed APP ICON and GUI Mock-up:

This is the currently proposed GUI, might change in future if needed.



Below is the icon for the app.



Software and Modules Required:

Software:

- Python 3
- PyCharm Community Edition
- JetBrains's Toolbox
- Git and GitHub
- Python Virtual Environment
- Internet connection for live API-based information

Python Libraries:

- PySide6 – development of the professional desktop graphical interface.
- Qt WebEngine – displaying interactive map content within the desktop application where required.
- NumPy – numerical calculations and vector operations.
- SciPy – additional scientific calculations where required.
- Pandas – processing weather, route and historical datasets.
- Requests / HTTPX – obtaining data from external APIs.
- Matplotlib – displaying graphs and analytical results.
- Scikit-learn – development and testing of machine-learning models.
- SQLite – storing local conditions, historical information and cached data.
- Pytest – automated testing of physics and application components.

Python's built-in modules such as math, json, date time and sqlite3 will also be used where appropriate.

Limitations:

- Live information depends on the availability, accuracy and usage limits of external APIs.
- Open/free services may have rate limits and fair-use restrictions.
- Weather information may not always accurately represent conditions at a specific road segment.
- Rural roads may contain local conditions that are unavailable through online data sources.
- Such local conditions may therefore require optional user-provided information.
- Machine-learning performance will depend on the quantity and quality of available historical data.

- Environmental conditions can change rapidly, so predictions cannot always be completely accurate.
- Some advanced features may need to be simplified depending on available free/open data sources.
- The application provides analytical route recommendations and is not a guarantee of road safety.

Bibliography:

1. Python Software Foundation. (2024). *Python 3.12.3 documentation*. <https://docs.python.org/3/>
2. Qt Project. (2024). *PySide6 documentation*. <https://doc.qt.io/qt pyside6/>
3. OpenStreetMap. (2024). *OpenStreetMap documentation*. <https://docs.openstreetmap.org/>
4. NumPy. (2024). *NumPy documentation*. <https://numpy.org/doc/stable/>
5. Pandas Development Team. (2024). *Pandas documentation*. <https://pandas.pydata.org/docs/>
6. SciPy. (2024). *SciPy documentation*. <https://docs.scipy.org/doc/scipy/>
7. Matplotlib. (2024). *Matplotlib documentation*. <https://matplotlib.org/stable/>
8. Scikit-learn Development Team. (2024). *Scikit-learn documentation*. <https://scikit-learn.org/stable/>
9. Pytest Development Team. (2024). *Pytest documentation*. <https://docs.pytest.org/>
10. SQLite. (2024). *SQLite documentation*. <https://www.sqlite.org/docs.html>
11. Arora, S. (2026). *Computer science with Python: Textbook for Class 11* (15th ed.). Dhanpat Rai & Co. (P) Ltd.

•